

The C++ programming language

December 4, 2025
Ramasamy Kandasamy

Contents

1	Language	2	5.9 Queue	11	8	Variable nomenclature for competitive programming	13
1.1	Fundamentals	2	5.10 Dequeue	11			
1.2	Expressions	2	5.11 Heap	11			
1.3	Declarations	3	5.12 Priority queue	11			
1.4	Initialization	3	5.13 Iterators	11			
1.5	Functions	4	5.14 Representing graphs	11			
1.6	Statements	4	5.15 Adjacency list	11			
1.7	Classes	5	5.16 Adjacency matrix	11			
1.8	Templates	6	5.17 Edge list	11			
1.9	Miscellaneous	7	6	Code snippets, etc.,	11		
2	Devtools	7	6.1 GCD:	11			
2.1	G++	7	6.2 Middle of an array	11			
2.2	GDB	7	6.3 Decimal to binary	11			
3	Input output	8	7	Previous Mistakes:	12		
3.1	File Access	8	7.1 Using proper variable types	12			
3.2	Reading and writing a file	8	7.2 Be careful of unsigned int	12			
3.3	Character I/O	8	7.3 Upper bound and lower bound	12			
3.4	Line I/O	8	7.4 Loops	12			
3.5	printf and scanf	8	7.5 Bugs in compiler directives	12			
3.6	cin and cout	8	7.6 Bugs due to augmented data	12			
3.7	Miscellaneous	8	7.7 min_element and max_element	12			
4	Libraries	9	7.8 map vs multiset, NOT exactly a mistake	12			
4.1	Character class tests: <cctype>	9	7.9 Math errors	12			
4.2	String functions: <cstring>	9	7.10 Not accessing out of bound elements	12			
4.3	Mathematical function: <cmath>	9	7.11 Order of precedence and brackets	12			
4.4	Utility Functions: <cstdlib>	9	7.12 Using braces appropriately in conditionals and loops	12			
4.5	Algorithms	9	7.13 Errors due to 0-based and 1-based indexing	12			
5	Data structures	10	7.14 Error while using obj.size() as limit in a loop	12			
5.1	Pair	10	7.15 Loop condition dependent on string::find	12			
5.2	Tuple	10	7.16 Semicolon typo	12			
5.3	Vectors	10	7.17 Bugs due to using infinity	12			
5.4	String	10	7.18 No code after return	13			
5.5	Set	10	7.19 Beware when using logical operators with functions	13			
5.6	Bit set	10	7.20 Be careful with char vs int	13			
5.7	Map	10	7.21 Be careful about pass by reference.	13			
5.8	Stack	11	7.22 a = a + 1 vs a += 1	13			
			7.23 Failure to initialize all the variables	13			

1. Language

1.1. Fundamentals

Pre-processor

Pre-processor directive always start with a '#'.

- `#include filename`
- `#define NAME replacement text`
NAME is replaced with *replacement text*
- `#define token(arg1,arg2) statement`
Defining a macro. Eg:
`#define max(A,B) ((A) > (B) ? (A) : (B))`
- `#undef NAME`
Nullifies existing definition of NAME.
- `#if, #elif, #else, #endif`
- `#ifdef and #ifndef`
Used in header-guards. Eg:

```
#ifndef HEADER_FILE_NAME
#define HEADER_FILE_NAME
...
#endif
```
- `#pragma once`: Single-inclusion header guard.
Widely supported, but not guaranteed.
- `#error "message"`: Emit a compile-time error.
- `#warning "message"`: Emit a compile-time warning (compiler extension).
- `#line N "file"`: Change reported line/file in diagnostics.
- Built-ins: `__FILE__, __LINE__, __DATE__, __TIME__, __cplusplus`(C++ version).

1.2. Expressions

Value Categories

lvalue	Has name. Can take address.
prvalue	Temporary value. No name. <i>Guaranteed copy elision.</i>
xvalue	Expiring. Eg: <code>std::move(x)</code> .
glvalue	<code>lvalue ∪ xvalue</code> .
rvalue	<code>prvalue ∪ xvalue</code> .
L-value reference	Refers to an lvalue. Eg: <code>int& ref = var;</code>
R-value reference	Refers to an rvalue. Eg: <code>int&& rref = 5;</code>

Exception: R-value can bind to const l-value reference.
`const int& ref = 5;`

Some exceptions:

- R-value can bind to const l-value reference.
Eg: `const int& x = 42;`
- String literals are l-values.

Literals

Boolean	<code>true</code> <code>false</code>	
Integer	<code>42</code> <code>4'200'000</code> <code>0b101010</code> <code>052</code> <code>0x2a</code>	Decimal. Decimal with separators. Binary. Octal. Hexadecimal.
Float	<code>3.14</code> <code>3.14f</code> <code>6.626e-34</code>	Double. Float. $x \times 10^y$
Character	<code>'a'</code> <code>'\n'</code> <code>'\13'</code> <code>u8'a'</code>	Ordinary literal. Using escape seq. UTF-8.
String	<code>"Hello"</code>	String literal.
Pointer	<code>nullptr.</code>	

NOTE: Literals are prvalues.

Exception: String literals are lvalues.

`std::initializer_list`

Constructed automatically from braced-init-list.

Eg:

```
Foo foo({1, 2, 3});
print_list({1, 2, 3});
```

Passed as argument to functions.

Eg:

```
void print_list(std::initializer_list<int> list);
Foo(std::initializer_list<int> list);
```

Access elements using iterators or range-based for loop.

Arithmetic and logical operation

	Operators	Associativity
1	<code>() [] -> .</code>	left to right
2	<code>! ~ ++ -- + - * & (type) sizeof</code>	right to left
3	<code>* / %(remainder)</code>	left to right
4	<code>+ -</code>	left to right
5	<code><< >></code>	left to right
6	<code>< <= > >=</code>	left to right
7	<code>== !=</code>	left to right
8	<code>&</code>	left to right
9	<code>^</code>	left to right
10	<code> </code>	left to right
11	<code>&&</code>	left to right
12	<code> </code>	left to right
13	<code>?:</code>	left to right
14	<code>= += -= *= /= %= &= ^= = <<= >>=</code>	right to left
15	<code>,</code>	left to right

Selected operators

`>>` Bitwise right shift operator.
`a = b >> 1` equivalent to dividing b by 2.

`<<` Bitwise left shift operator.
`a = b << 1` equivalent to multiplying b by 2.

`?:` Ternary conditional operator.
`x = (condition)? expr1: expr2;`
`expr1 → condition == true.`
`expr2 → condition == false.`

Type casting

`const_cast` Modifies `const/volatile` qualifiers.
Modifying a `const` object leads to UB.

`static_cast` Compile-time downcast.
Not safe.

`dynamic_cast` Safe downcast.
Eg: `Base* b = static_cast<Base*>d;`
when invalid b is `nullptr.`
Eg: `Base& b = static_cast<Base&>d;`
when invalid throws `std::bad_cast.`

`reinterpret_cast`

1.3. Declarations

<delc-type-specifier> <declarator>

Examples:

```
int x; // <int> <x>
int *x; // <int> <*x>
const int *x; // <const int> <*x>
int *const x; // <int> <const *x>
const int *const x; // <const int> <const *x>
```

Declarator

Tells the compiler about the name of the variable, its type (pointer, array, function etc).
Could also have cv qualifiers (const, volatile).
Eg: int *const p;
Here const is part of the declarator.

decl-type-specifiers

Fundamental types	
char	
int	
float	
double	
void	
Common aliases	
size_t	
char_t	
int8_t	
int64_t	
uint32_t	
NOTE: std::size_t, int8_t are preferred over size_t, int8_t etc. Why?	
register	Ignored by modern compilers.
mutable	Removes const-ness from a const object member variable.
inline	Relaxes ODR. NOTE: Global linkage.
static	Specifies internal linkage. <i>Local:</i> initialized only once, retains value between calls. <i>Global:</i> visible only in the current TU. Different from inline. <i>See "Class" for static members.</i>
extern	Specifies external linkage. Eg: extern int x; // x is now available in other TUs.
thread_local	Specifies thread local storage duration. Eg: thread_local int x;

typedef	typedef <type> <alias>; Eg: typedef long long int ll;
using	using <alias> = <type>; Eg: using ll = long long int; using supports template aliase, but typedef does not.
namespace	Declares a namespace. Eg: namespace Foo { ... } Alias: namespace Bar = Foo;
auto	Type deduction based on initializer. Eg: auto x = 10; // x is int
decltype	Deduces type of an expression. Eg: decltype(x) y; // y has same type as x decltype is exact, unlike auto. Eg: auto doesn't preserve references, const, arrays decay to pointers etc.
static_assert	Compile time assertion. static_assert(<bool-constexpr>) static_assert(<bool-constexpr>, " <message> ")

enum, enum class, union

enum Eg:

```
enum Color { RED,          // 0
             GREEN = 5,    // 5
             BLUE          // 6
};
```

- No scope:Eg: Color c = RED;.
- Implicitly converts to int. Eg: int x = RED;.

enum class (scoped enum) Eg:

```
enum class Color {
    Red,          // 0
    Green = 3,    // 3
    Blue          // 4
};
```

Eg usage: Color c = Color::Red;.
Union
Eg:

```
union Data {
    int i;
    float f;
};
```

1.4. Initialization

Fundamental Types

```
int a;          // Garbage value.
int a{};        // a = 0.
int a{1};       // a = 1;
int a = b;      // Copy intialization.
```

Array types

```
int a[3];          // Garbage value.
int a[3]{};        // a = {0, 0, 0}
int a[3]{1};       // a = {1, 0, 0}
int a[3]{1, 2, 3}; // a = {1, 2, 3}
```

Aggregate Types

Aggregates: Arrays, "Simple" structs/classes without Ctors.
Consider:

```
class Foo {
public:
    int x;
    double y;
};
```

```
Foo foo;
Foo foo{};          // foo.x = 0; foo.y = 0.0;
Foo foo{1};         // foo.x = 1; foo.y = 0.0;
Foo foo{1, 2};      // foo.x = 1; foo.y = 2.0;
```

Class Types with Constructors

```
Foo foo;            // Default Ctor.
Foo foo();           // Default Ctor.
Foo foo{};          // Default Ctor.
Foo foo{bar};       // Copy Ctor. Not blocked by explicitly.
Foo foo(bar);       // Copy Ctor. Not blocked by explicitly.
Foo foo = bar;      // Copy Ctor. Blocked if explicit.
Foo foo(1, 2);      // Parameterized Ctor.
Foo foo{1};         // Ctor with std::initializer_list.
Foo foo{1, 2};      // Ctor with std::initializer_list.
Foo foo({1, 2});    // Ctor with std::initializer_list.
```

1.5. Functions

- **Definition :** `T foo(args) {...}`.
- **Declaration :** `T foo(args);`.
- **Variadic Arguments:** `T foo(int n, ...)`.
C-style. Not safe. Use variadic templates.

The main() function

```
main(int argc, char *argv[]) {  
    ...  
}  
argc      No. of args including the program name.  
argv[0]    Pointer to executable name.  
argv[i]    Pointer to ith argument.
```

Lambda function

Usage: `[<capture>](<params>)-><return-type>{<body>}`

`<capture>` Specifics

`[]` No variables captured.

`[=]` All variables captured by value.

`[&]` All variables captured by reference.

`[x, &y]` x captured by value, y by reference.

`[=, &y]` All variables except y by value, y by reference.

NOTE: Not specifying return type is not compile time error and ofter compiler can deduce it. But, it is good practice to specify it, especially for complex lambdas.

Eg:

```
// Sort descending using lambda  
sort(nums.begin(), nums.end(), [](int a, int b) -> bool {  
    return a > b;  
});
```

Passing functions as arguments

1. Function pointers.

Eg:

```
int foo(int a, int b, int (*bar)(int, int)) {  
    return bar(a, b);  
}  
// Function call.  
foo(5, 3, add);
```

Doesn't work with lambdas with captures.

2. `std::function`

```
int foo(int a, int b, std::function<int(int, int)> bar) {  
    return bar(a, b);  
}  
// Function call.  
foo(5, 3, add);
```

3. Templates

```
template <typename bar>  
int foo(int a, int b, bar bar) {  
    return bar(a, b);  
}  
// Function call.  
foo(5, 3, add);
```

Zero overhead

1.6. Statements

If-else	<pre>if (<i>expression</i>) <i>statement</i> else if (<i>expression</i>) <i>statement</i> : : else <i>statement</i></pre>
Switch	<pre>switch (<i>expression</i>) { case <i>const-expr</i>: <i>statements</i>; break; case <i>const-expr</i>: <i>statements</i>; break; default: <i>statements</i>; break; } Without break statement the execution will <i>fall through</i> all the cases. This can be used as: switch (<i>expression</i>) { case 1: case 2: case 3: group = 0; break; case 4: case 5: group = 1; break; default: group = -1; break; }</pre>
While	<pre>while (<i>expression</i>) <i>statement</i></pre>
For	<pre>for (expr1; expr2; expr3) <i>statement</i> This for loop is equivalent to: expr1; while (expr2) { <i>statements</i>; expr3; }</pre>
Do While	<pre>do <i>statement</i> while (<i>expression</i>);</pre>
Break	break : Exit the the loop or control. Works for while, for, do while and switch

Continue	continue: Continue next iteration. Works for while, for, do while If switch is within a loop and if continue is used inside switch and if it is executed, then the loop skips to next loop.
Goto and label	Example: if (disaster) { goto error; } ... error: { statement }

Shorter alternative for if-else statement:
x = <cond-expr> ? <out-if-true> : <out-if-false>;
Eg: char is_pos = (n > 0) ? 'y' : 'n';

1.7. Classes

Eg:

```
class Point {
public:
    Point(int x, int y) : x_(x), y_(y) {}
private:
    int x_;
    int y_;
};
```

Inheritance Behaviour

Simple rule

1. Private members are never inherited.
2. derived-type = min(base-type, inheritance-type).

Member / Inheritance	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	NA	NA	NA

Members Variables

- this - pointer to the current object.
- static - shared across all objects of the class.

Member Functions

Rule-of-5 members

1. Copy Ctor

```
// Pass by ref to avoid recursion.
Vec::Vec(const Vec &other) {
    n_ = other.n_;
    data_ = new int[n_];
    for (int i = 0; i < n_; i++) {
        data_[i] = other.data_[i];
    }
}
```

2. Move Ctor

```
Vec::Vec(Vec &&other) noexcept {
    n_ = other.n_;
    data_ = other.data_;
    other.n_ = 0;
    other.data_ = nullptr;
}
```

3. Copy Assignment Operator

```
Vec& Vec::operator=(const Vec &other) {
    if (this != &other) { // Self-assignment check.
        delete[] data_; // Free existing resource.
        n_ = other.n_;
        data_ = new int[n_];
        // Deep copy.
        for (int i = 0; i < n_; i++) {
            data_[i] = other.data_[i];
        }
    }
    return *this; // To permit chained assignments.
}
```

4. Move Assignment Operator

```
Vec& Vec::operator=(Vec &&other) noexcept {
    if (this != &other) { // Self-assignment check.
        delete[] data_; // Free existing resource.
        n_ = other.n_;
        data_ = other.data_;
        other.n_ = 0;
        other.data_ = nullptr;
    }
    return *this; // To permit chained assignments.
}
```

5. Destructor

```
Vec::~Vec() {
    delete[] data_;
}
```

Others

6. Default Ctor Eg: Foo::Foo()

7. Converting Ctor

Ctor with single argument or multiple arguments with default values.

```
Say: Foo::Foo(const Bar&);

Foo f = bar; // Implicit conversion from Bar to Foo.

explicit Foo(const Bar&); // Prevents implicit conversion.
```

8. Static methods

Belong to class, not object.
No this pointer.

Compilers provided default implementation for 1-6 methods.

- **delete:** Prevents default generation.
- **default:** Makes the intent clear.
Forces default generation?
- **explicit:** Prevents implicit conversions for converting ctors.

Virtual Methods and Abstract Class

- **Virtual method:** Eg: virtual void Animal::speak();
- **Pure virtual method:** Eg: virtual void Animal::speak() = 0;
- **Abstract class:** Class with at least one pure virtual method. Cannot instantiate objects of abstract class.

Virtual vs normal methods:

```
Animal* a = new Dog();
a->speak();
```

- **speak** is virtual: Dog's speak() called.
- **speak** is normal: Animal's speak() called.

override & final

Specifiers at the end. Relative order between them doesn't matter.
override:
Tells a method is meant to override a base class method.
Compiler error if it doesn't.
Eg:

```
Base: void Animal::speak() const;
Derived: void Dog::speak(); // Compiles.
Unexpected runtime behaviour:
a->speak(); // Base class speak.
Derived w/ override:
void Dog::speak() override; // Compiler error.
```

final:
Method not supposed to be overridden in derived classes.
Compiler error if it is.
Eg:

```
Base: void Dog::speak() override final;
Derived:
void BorderCollie::speak() override; // Compiler error.
```

1.8. Templates

Template Parameter Kinds

Type	typename T or class T Accepts any type as argument.
Non-type	int N, size_t N, auto N, etc. Accepts compile-time constant values. Types: integral, pointers, references, nullptr_t, enums, float (C++20).
Template template	template <typename> class C template <typename> typename C (C++17+) Accepts template as argument.

Template Parameter Syntax

Basic	typename T int N template <typename> class C
With default	typename T = int int N = 10 template <typename> class C = std::vector
Variadic	typename... Args int... Ns template <typename> class... Containers

NOTE: **typename** vs **class** for type parameters are interchangeable. **typename** is preferred (more explicit). Also, **typename** has another use: disambiguating dependent types. Eg: **typename T::value_type x;**

Template Specialization

Full/Explicit Specialization

Provide custom implementation for specific type(s).

Eg:

```
// Primary template
template <typename T>
class MyClass { /*...*/ };

// Full specialization for bool
template <>
class MyClass<bool> { /*...*/ };
```

Partial Specialization

Specialize for a subset of types. **Only for class templates, not functions.**

Eg:

```
// Primary template
template <typename T, typename U>
class Pair { /*...*/ };

// Partial specialization: both types same
template <typename T>
class Pair<T, T> { /*...*/ };

// Partial specialization: pointer types
template <typename T, typename U>
class Pair<T*, U*> { /*...*/ };
```

Full specialization	template <> class C<int> {...} All parameters specified. Works for classes and functions.
Partial specialization	template <typename T> class C<T*> {...} Some parameters remain generic. Only for class templates.

Parameter Packs

Syntax

Declaration	typename... Args int... Ns
Expansion	Args... expands to T1, T2, T3, ... func(Args...)... expands to func(T1), func(T2), ... sizeof...(Args) gives pack size.

Example: Compile-time Fibonacci sequence

```
// Fibonacci calculation
template <int N>
struct Fib {
    static constexpr int value =
        Fib<N-1>::value + Fib<N-2>::value;
};

template <>
struct Fib<0> { static constexpr int value = 0; };
template <>
struct Fib<1> { static constexpr int value = 1; };

// Variadic template to hold sequence
template <int... Ns>
struct FibSeq {
    static constexpr int values[] = { Fib<Ns>::value... };
};

// Usage
constexpr int fibs[] = FibSeq<0, 1, 2, 3, 4, 5>::values;
// fibs = {0, 1, 1, 2, 3, 5}
```

Pack expansion Fib<Ns>::value... expands to Fib<0>::value, Fib<1>::value, ..., Fib<5>::value

1.9. Miscellaneous

2. Devtools

2.1. G++

Usage:

```
g++ option hello.cpp -o hello
```

```
g++ hello.cpp -o hello
```

```
g++ -O3 hello.cpp -o hello
```

Options:

-o	Output file name.
-I	Include additional directories in the search path. Eg: <code>g++ -I /home/user/libs foo.cpp</code>
-O<level>	Optimization level, starts from 0. Eg: -O3
-g	Creates breakpoints for debugging using GDB.
-lm	Missing link. Used to link certain libraries like <code>math.h</code> .
-E	Output preprocessed but uncompiled code.
-S	Compile but do not assemble.
-c	Compile and assembly, but do not link.

2.2. GDB

To run: `gdb hello`

<code>break</code>	Set break point. Eg: <code>break function</code> or <code>break line number</code> .
<code>run</code>	Run the program. The program stops at every break point.
<code>next</code>	Run until next breakpoint.
<code>print</code>	Eg: <code>print i</code> or <code>print &i</code>
<code>sizeof</code>	Eg: <code>print sizeof(i)</code>
<code>&i</code>	Address of i.
<code>*j</code>	Content of memory location j.
<code>ptype</code>	get type of a variable. Eg: <code>ptype(i)</code>
<code>set var</code>	Reassign variable. Eg: <code>set var i = 1</code>
<code>disassemble</code>	Use after break and run. Gives assembly code.

Accessing memory location using x

Usage: `x/nfs`. `nfs` describes the format.

n - Number of units to display.

f - Number format.

s - Size of each unit.

x	Hex.
o	Octal.
t	Binary.
d	Decimal.
u	Unsigned decimal.
i	Instruction.
c	Character.
s	String.

b	byte.
h	Halfword.
w	Word.
g	Giant.

3. Input output

3.1. File Access

```
Format:
FILE *fp;
FILE *fopen(char *name, char *mode);
int fclose(FILE *fp);
fp = fopen(name, mode);
Allowable modes include:
"r"   read.
"w"   write.
"a"   append.
"b"   binary files: "rb", "wb", "ab".
```

NOTE: **Difference between rb and r etc:**
While using just "r" might work in writing binary files, it might cause trouble due misinterpretation of special characters like LF and CR.
See: <https://stackoverflow.com/q/2174889/5607735>

Standard I/O

Can be treated in the same way as file pointer.
stdin, stdout, stderr

3.2. Reading and writing a file

```
size_t fread(const void* ptr, size_t size,/
size_t count, FILE* stream);

size_t fwrite(const void* ptr, size_t size,/
size_t count, FILE* stream);
```

3.3. Character I/O

```
getchar    Takes input from standard input.
            int getchar()
            Equivalent togetc(stdin)
putchar    Output to standard output.
            int putchar(int)
            Equivalent toputc(c), stdout
getc       int getc(FILE *fp), Eg: getc(stdin)
putc      int putc(int c, FILE *fp)
ungetc
```

3.4. Line I/O

```
gets       Reads until EOF.
            gets deletes terminal \n
puts       Writes line to stdout.
            puts adds terminal \n
fgets      char *fgets(char *line, int maxline, FILE *fp)
fputs      int *fputs(char *line, FILE *fp)
getline    Eg: getline(cin, s). Where s is a string.
```

NOTE: Never use gets
gets does not check for buffer overrun, and keeps reading until it encounters new line or EOF. It has been used to break computer security.

3.5. printf and scanf

Conversion formats for printf and scanf	
Character	Argument type; Printed as
d,i	int; decimal number
o	int; unsigned octal number (without leading zero)
x,X	int: unsigned hexadecimal number (without a leading 0x or 0X), using abcdef or ABCDEF.
u	int; unsigned decimal number.
zu	size_t.
c	int; single character.
s	char *; Print string, scan word. NOTE: scanf reads only a word and not the entire string.
f	double; [-]m.dddddd.
e,E	double; [-]m.dddddd±xx, or [-]m.dddddd±xx.
g,G	double; %e or %E if exponent is < -4 or >= precision, else use %f.
p	void *; pointer.
%	no argument; Print %.

Between % and conversion character there may be in order:

- Minus sign: Left justification.
- Number: Minimum field width.
- Period: Separates field width from precision.
- Number: Precision.
For float: number of digits after decimal.
For int: minimum number of digits.
For string: maximum number of characters to be printed.
- h: for short integer. l: for long integer.

Formatting rules

Examples	Wildcard Here precision can be specified dynamically. Eg: printf("%*d", 6, foo);, this is equivalent to printf("%6d",foo); Integers %d print as decimal integer. %6d print as decimal integer, at least 6 characters wide. Floating point numbers %6f print as floating point. %.2f print as floating point, 2 characters after decimal point. %.6.2f print as floating point, at least 6 characters wide and 2 characters after decimal point. String: example hello, world (12 chars). %s: :hello,_world: %10s: :hello,_world: %.10s: :hello,_wor: %-10s: :hello,_world: %.15s: :hello,_world: %-15s: :hello, world_:_: %15 .10s: :_ _ _ _hello, wor: %-15 .10s: :hello,_wor_ _ _ _:
----------	---

Printf and variants

- int printf(char *format, arg1, arg2, ...);
Print output to stdout.
- int sprintf(char *string, char *format arg1, arg2, ...);
Print output to string.
- int fprintf(FILE *fp, char *format arg1, arg2, ...);
Print output to file pointed by the file pointer fp.

```
Example:
printf("%s\n", string_var);
printf("%s", "hello, world");
printf("square of n is %d\n", n_square);
```

Scanf and variants

- int scanf(char *format, arg1, arg2, ...);
Scan input from stdin.
- int sscanf(char *string, char *format arg1, arg2, ...);
Scan input from string.
- int fscanf(FILE *fp, char *format arg1, arg2, ...);
Scan input from the file pointed by the file pointer fp.

NOTE1: unlike printf, in scanf the variable are pointers.
NOTE2: In scanf %s reads a word and not the entire string.
Also see: <https://stackoverflow.com/a/1248017/5607735>

```
Examples:
scanf("%d", &n);
sscanf(string, "%d", &n);
fscanf(fp, "%d", &n);
```

3.6. cin and cout

```
cin    Read input until space, or tab or LF.
cout   Output.
```

NOTE:
• Use the following to increase speed of I/O.
ios_base::sync_with_stdio(false);
cin.tie(nullptr); cout.tie(nullptr);
• Use cin.ignore() after cin and before using getline.

3.7. Miscellaneous

```
int fflush(FILE *stream)
Causes any buffered but unwritten data to be written. The effect is undefined on an input stream.
int fclose(FILE *stream)
Flushes any unwritten data for stream, discards any unread buffered input.
```


4. Libraries

Usually in Linux, the library header files are stored in `/usr/include`

NOTE: Use `#include <bits/stdc++.h>` to include all standard libraries.

4.1. Character class tests: `<cctype>`

```
isalpha(c)
isupper(c)
islower(c)
isdigit(c)
isalnum(c)
isspace(c)    true for ASCII codes: 9-13 & 32.
toupper(c)
tolower(c)
```

4.2. String functions: `<cstring>`

```
strcat(s,t)    Concatenate t to end of s
strncat(s,t,n)
strcmp(s,t)
strncmp(s,t,n)
strcpy(s,t)    Copy t to s
strncpy(s,t,n)
strlen(s)
strchr(s,c)    Return pointer to first c
strchr(s,c)    Return pointer to last c
```

4.3. Mathematical function: `<cmath>`

```
sin(x)
asin(x)
hsin(x)
exp(x)
log(x)
log10(x)
pow(x,y)       $x^y$ 
sqrt(x)
ceil(x)
floor(x)
abs(x)        Abs. val. of x. Returns integers types.
fabs(x)       Absolute value of x. Returns double
frexp        frexp(int x, int *exp). Returns significand ( $y$ ) in
              the range  $[0.5, 1)$  and the stores the exponent in exp,
              such that  $x = y * e^{exp}$ 
ldexp        ldexp(double y, int exp). Inverse of frexp.
modf         double modf(x, double *ip). Splits  $x$  into integer
              and fractional parts. Returns the fractional part and
              stores integer parts at ip.
fmod(x,y)    Floating point remainder of  $x/y$  with the same sign as
               $x$ .
```

4.4. Utility Functions: `<cstdlib>`

```
system(char *s)
    Run system commands.
    Eg: system("date")
```

String `int`-`int` numbers

String to numbers

From C library `int atoi(char *s)`.

Variants: `atof`, `atol`, `atod`

From C++ library `int stoi(string s, size_t *p = 0, int base = 10)`.

Variants: `stol`, `stod`, `stof`, `stold`

Eg:

```
int n = stoi("1234"); // n = 1234.
int n = stoi("12", 0, 16); // n = 18.
int n = stoi("A", 0, 16); // n = 10.
```

Number to string

`to_string` Eg: `string s = to_string(1234);` `s = "1234"`.

Memory management

- `void *calloc(size_t nobj, size_t size)`
Return pointer to array of `nobj` of size `size`.
The space is initialized to 0.
- `void *malloc(size_t size)`
Returns pointer to an object of size `size`.
The space is uninitialized.
- `void *realloc(void *p, size_t size)`
Change size of the object pointed to by `p` to `size`.
Return pointer to new space.
- `free (void *p)`
Deallocates space point to by `p`.

Memory management in C++

- `void* operator new(size)`
Eg: `myClass* p1 = new myClass;`
`myClass* p1 = new(sizeof(myClass));`
- `void delete(void* ptr)`
Eg: `delete p1;` `delete(p1);`

limits

Examples:

- `numeric_limits<int>::min()`
- `numeric_limits<int>::max()`
- `numeric_limits<double>::min()`
- `numeric_limits<double>::infinity()`

4.5. Algorithms

Sort and search

Search and sort from C

- `void *bsearch(const void *key, const void *base, size_t n, size_t size, int (*cmp)(const void *key1, const void *datum))`
Searches `base[0] ... base[n-1]` for `key`.
Comparison function must return negative if its first argument (search key) is less than its second argument (a table entry) and so on.
- `qsort(void *base, size_t n, size_t size, int (*cmp)(const *void, const *void))`

Search and sort from C++

```
Compare functions:
bool compare(x, y){
    return(x strictly precedes y)
}
```

In all the below, `a` could be an iterator or a pointer and all of them can use custom compare functions.

- `y = lower_bound(a, a+n, x)`
Returns an iterator to the first element whose value is $\geq x$.
- `y = upper_bound(a, a+n, x)`
Returns an iterator to the first element whose value is $> x$.
- `y = equal_range(a, a+n, x)`
Returns a pair of iterators pointing to the upper bound and lower bound of x .
- `y = equal_range(a, a+n, x, compare_function)`
Returns a pair of iterators pointing to the upper bound and lower bound of x .
- `sort(it1, it2, compare_function);`
- `stable_sort(it1, it2, compare_function);`
- `is_sorted(it1, it2, compare_function);`
- `is_sorted_until(it1, it2, compare_function);`
Returns the iterator to the element until which the array is sorted.
- `find_if(start_it, end_it, func)`
Returns iterator to the first element in the range `[start_it, end_it)` for which `func` returns true.
- `find_if_not(start_it, end_it, func)`
Returns iterator to the first element in the range `[start_it, end_it)` for which `func` returns false.

Min Max

- `max(a, b, comp); min(a, b, comp);`
- `max({a1, a2, a3}, comp); min({a1, a2, a3}, comp);`
- `max_element(a, a+n, comp);`
Returns the iterator for the largest element in `a`. If there are multiple largest element, returns the iterator/pointer for the first one.
- `min_element(a, a+n, comp);`

Merge

- `merge(InputIterator F1, InputIterator L1, InputIterator F2, InputIterator L2, OutputIterator R);`
Returns the iterator to the end of output.
NOTE: This operator does not allocate memory. The `OutputIterator R` should point to a vector with sufficient memory to store the concatenated vector.
- `set_intersection` Arguments and output are the same as that for `merge`.
- `set_union` Arguments and output are the same as that for `merge`.
- `set_difference` Arguments and output are the same as that for `merge`.
- `set_symmetric_difference` Arguments and output are the same as that for `merge`.

Hash Function

Can be used for custom types in `unordered_map` and `unordered_set`.

```
Usage:
hash<T> hasher;
size_t hasvalue = hasher(x); //Where x is an object of
type T
```

Eg:

```
hash<int> h; size_t x = h(10);
hash<string> h; size_t x = h("hello");
```

Custom hash function.

Custom hash function for a user defined type can be defined as follows:

```
class CustomHash {
public:
    size_t operator () (const pair<string, string>& p) const {
        hash<string> hasher;
        auto h1 = hasher(p.first);
        auto h2 = hasher(p.second);
        return h1 ^ h2; // Below for better heuristics.
    }
};
```

Heuristics to combine hash values.

$h2 = h2 \oplus (h1 \ll 1)$; $h2 \oplus = h2 + 0x9e3779b9 + (h1 \ll 6) + (h1 \gg 2)$;
`0x9e3779b9` is a prime number.

See: <https://stackoverflow.com/a/2595226/5607735>

Others

- `swap(T& a, T& b);`
- `reverse(Iterator first, Iterator last);`
- `next_permutation(a.begin(), a.end());`
- `prev_permutation(a.begin(), a.end());`
- `random_shuffle(a.begin(), a.end());`
Gives the next permutation for a given vector of integers.

5. Data structures

5.1. Pair

Eg: `pair<int, int> x = 1,2;`
`x.first` Returns 1.
`x.second` Returns 2.

5.2. Tuple

Fixed length collection of heterogenous values.

Eg: `tuple<int, string, double> t;`

Operations related to tuple

`get<i>` Return i^{th} value of t. Eg: `get<0>(t);`.
`make_tuple()` Returns tuple out of individual values.
Eg: `make_tuple(5, "Paul", 23.4)`.
`tie()` Unpacking tuple.
Eg:
`tie(n, s, x) = t;`
`tie(n, std::ignore, x) = t;`

5.3. Vectors

Definiton:

`vector<T> v;`

Element access

`operator[]`
`v.back()` Last element
`v.front()` First element

Modifiers

`v.push_back(5)` Add 5 to the vector. Similar to push in stack data structure.
`v.insert()` Inserts elements / range in v.
Eg: Append u to v.
`v.insert(v.end(), u.begin(), u.end());`
`v.emplace{iterator pos, Args}`. New element is constructed in place using `Args` and inserted at pos.
`v.emplace_back` `v.emplace_back{Args}`. New element is constructed in place using `Args` and inserted at the end.
IMPORTANT: Note the differences between `pushback`, `insert`, `emplace` and `emplaceback`.
`v.pop_back()` Remove element on top of the stack.
`v.erase()` Remove element / range from v.
`v.clear()` Clears all the elements in v.

Capacity

`v.size()` Returns the size of the vector.
`v.capacity()` Current memory allocated to v in terms of number of elements.
`v.max_size()` Maximum memory available for v.
`v.reserve()` Reserve a minimum size for v. Does not affect `v.size()`.
`v.shrink_to_fit()` Shrink the capacity to fit the size.
`v.empty()` Test if v is empty.

`tie` Can be used to unpack a vector.

Example: `tie(a, b, c) = v;`

5.4. String

A string is like a vector of characters.

Definition:

`string s = "Hello";`

String operations:

<code>a + b</code>	Concatenate.
<code>s.append(t)</code>	append t to s.
<code>s.compare(t)</code>	returns 0 if <code>s == t</code> , returns <code>< 0</code> if <code>s < t</code> , else returns <code>> 0</code> .
<code>s.substr(start, len)</code>	Select a substring.
<code>s.find(str2)</code>	find first occurrence of <code>str2</code> .
<code>s.find(str2, pos)</code>	Find first occurrence of <code>str2</code> by searching from the position <code>pos</code> .

5.5. Set

Definition

`set<type> s;`
`multiset<type> s;`
`unordered_set<type> s;`
`unordered_multiset<type, hasher> s;`
`hasher`: Refer Custom Hash Function.

Set operations

`s.insert(item);`
`s.erase(item);`
`s.count(item);`
`s.find(item);` Returns the iterator that points to `item`.
`s.insert()` Inserts elements in s.
Eg: Insert elements from t to s.
`v.insert(v.end(), u.begin(), u.end());`

`s.size()` Not available for `unordered_set` and `unordered_multiset`.

`s.lower_bound(x)` Returns iterator to the smallest element that is larger than or equal to x.

`s.upper_bound(x)` Returns iterator to the smallest element that is larger than x.

NOTE: erase method remove all instances of the item in the multiset.

5.6. Bit set

Definiton

`bitset<size> s;`

Examples:

`bitset<5> s; bitset<5> s(string("1100110"));`

Operations on Bit set:

`a & b` Bitwise AND.
`a | b` Bitwise OR.
`a ^ b` Bitwise EXOR.
`a` Bitwise negation.

5.7. Map

`map`: Balanced binary tree.

`unordered_map`: Hash list.

Definition:

`map<type1, type2> m;`

Eg: `map<string, int> m;`

`unordered_map<type1, type2, hasher> m;`

hasher: Refer Custom Hash Function.

Operations on map

```
m["banana"]    Retrive value associated with banana.
m.count("banana") Return 1 if the key is present else 0.
m.size();
```

5.8. Stack

```
Eg: stack<int> s;
    s.push(5)
    s.top()
    s.pop()
```

5.9. Queue

```
Eg: queue<int> q;
    q.push(5)    Adds element to the end.
    q.pop()      Removes first element.
    q.front()    Returns first element.
    q.empty()    True if q is empty.
```

5.10. Dequeue

```
Eg: dequeue<int> d;
    d.push_back(5)    Add to the top.
    d.push_front(2)   Add to the bottom.
    d.pop_back()      Remove from the top.
    d.pop_front()     Remove from the bottom.
```

5.11. Heap

Format:

Constructing a heap:

```
make_heap(iterator f, iterator s);
make_heap(iterator f, iterator s, comp);
```

```
pop_heap(iterator f, iterator s);
pop_heap(iterator f, iterator s, comp);
```

```
push_heap(iterator f, iterator s);
push_heap(iterator f, iterator s, comp);
```

NOTE:

f points to the first element and s points next to the second element.
f and s can be pointers when the heap is constructed over a array.

Compare function:

```
bool compare(x, y){
return(x < y); // For max heap
return(x > y); // For min heap
}
```

5.12. Priority queue

Format:

```
priority_queue(type, vector<type>, compare)
Max priority queue [Default]
Eg: priority_queue<int> q;
Min priority queue
priority_queue<int, vector<int>, greater<int>> q;
    q.push(3)
    q.top()    Returns the largest element.
    q.pop()    Removes the largest element.
```

Compare function for priority queues

```
class compare{
public:
    bool operator()(const T& x, const T& y) const {
        return(x < y); // Max priority queue.
        // return(x > y); // Min priority queue.
    }
}
```

5.13. Iterators

Iterators are like pointers but more specific to a collection.
Not applicable for `unordered_set`, `unordered_map` and `priority_queue`.

Example definition:

```
vector<int>::iterator it;
set<string>::iterator it;
```

```
s.begin()    Returns iterator pointing to the first element in the
              collection.
s.end()      Returns iterator pointing next to the last element in
              the collection.
```

Iterating over a map:

Here *it is a pair of the key and value. Eg: `pair<string, int> p = *it;`
Eg:

```
map<string, int>::iterator it = m.begin();
// Accessing key and value
string key = it->first;
int value = it->second;
```

Reverse Iterators

5.14. Representing graphs

5.15. Adjacency list

Eg:
`vector<int> adj[N]` // Unweighted graph.
`vector<pair<int,int>> adj[N]` // Weighted graph.

5.16. Adjacency matrix

Eg: `int adj[N][N];`

5.17. Edge list

Eg:
`vector<pair<int,int>> edges` Unweighted graph.
`vector<tuple<int,int,int>> edges` // Weighted graph.

6. Code snippets, etc.,

6.1. GCD:

```
int gcd(int x, int y){
    return y ? gcd(y, x%y) : x;
}
```

6.2. Middle of an array

n: length of array

0-based indexing

if n is odd: $\left\lfloor \frac{n}{2} \right\rfloor$
if n is even: $\left\lfloor \frac{n}{2} \right\rfloor - 1$ and $\left\lfloor \frac{n}{2} \right\rfloor$

1-based indexing

if n is odd: $\left\lceil \frac{n}{2} \right\rceil$
if n is even: $\left\lceil \frac{n}{2} \right\rceil$ and $\left\lceil \frac{n}{2} \right\rceil + 1$

6.3. Decimal to binary

```
int n;
cin >> n;
auto a = bitset<64>(n);
```

a[0] contains zeroth bit and so on.

7. Previous Mistakes:

7.1. Using proper variable types

Eg: using `int` instead of `long` `int` would cause erroneous output.

7.2. Be careful of unsigned int

Eg

```
unsigned x; cin >> x;
while(x >= 0){
    // Do something
}
```

The above will never terminate, since `x` never becomes negative.

7.3. Upper bound and lower bound

Check for edge cases. While searching for `x`, if `x` is not found, then `lower_bound` points to an element larger than `x`. The following might be helpful.

```
if(lower_bound > begin && *lower_bound > x) lower_bound--;
if(lower_bound == end) lower_bound--;
if(upper_bound == end) upper_bound--; // Might be
necessary sometimes.
```

7.4. Loops

- Using variable in boundary condition. Eg:

```
// value of x changes within the loop.
for(int i = 0; i < (d[x] - c); i++) // Dangerous !!
```

```
k = d[x] - c;
for(int i = 0; i < k; i++) //
```

- When the variable in boundary condition is updated by multiplication.
In the following loop when `n > 1` and `a = 1`, this loop won't terminate.

```
int x = 1;
while(x < n){
    x = a * x;
}
```

7.5. Bugs in compiler directives

In Bioinformatics contest 2021, a bug was because I use `bitset<M>`, where `M` was specified as compiler directive. While debugging I did not pay attention to `#define` statements.

Lesson: While debugging pay attention to the entire code, particularly compiler directives. They are particularly important because they are hidden and not directly noticeable in the code.

7.6. Bugs due to augmented data

Ref: ABC_207 problem C. There were two vectors, `v` and `t` associated by index. The vector `v[i]` corresponds to `t[i]`. In this problem, I sorted `v` independently from `t`. Thus this broke the association between `v` and `t` resulting in erroneous results.

7.7. min_element and max_element

Do not use these two functions to find minimum and maximum elements in an unsorted array. Read documentation to know more about them.

7.8. map vs multiset, NOT exactly a mistake

In one problem I used multiset to keep count of some values. It was very slow compared to using map to keep count of those values.

7.9. Math errors

Division by zero and logarithm of zero.

7.10. Not accessing out of bound elements

```
if(a[i] == 0) {do something;} // Wrong.
if(i < n && a[i] == 0){do something;}// Correct.
```

7.11. Order of precedence and brackets

Example:(CRF-732: 1546C)

```
if(l_pos[k] - l_pos[k-1] % 2 == 1){do something}// Wrong;
if((l_pos[k] - l_pos[k-1]) % 2 == 1){do something}// Correct;
```

7.12. Using braces appropriately in conditionals and loops

Example:(CRF-732: 1546C)

```
// Wrong if do_2 depends on x :
if(x) do_1;
    do_2;
```

```
// Correct if do_2 depends on x :
if(x){ do_1;
    do_2;
}
```

7.13. Errors due to 0-based and 1-based indexing

The data in most programming contests use 1-based indexing. Error could arise while reading or using such data.

Eg:

```
for(int i = 0; i < n; i++){
    int temp; cin >> temp;
    a[temp]++;
}
```

Here if `a` is 0-based and input `temp` is 1-based, then error will arise. The following is correct.

```
for(int i = 0; i < n; i++){
    int temp; cin >> temp;
    a[temp-1]++;
}
```

7.14. Error while using obj.size() as limit in a loop

`Obj.size()` return a value of type `size_t` which (probably) unsigned long long int. Therefore it inherits the same dangers associated with using `unsigned int`(13.2). Eg: Here if `v.size()` is zero, `v.size()-1` will not return -1, rather it would return the largest long long.

```
for(int i = 0; i <= v.size()-1; i++){}
```

Probable a correct way to write the above code is as follows:

```
for(int i = 0; i <= (int)v.size()-1; i++){}
```

Another example:

```
for(int i = r.length()-2; i >=0; i++){}
```

Correct version.

```
int rl = r.length();
for(int i = rl-2; i >=0; i++){}
```

7.15. Loop condition dependent on string::find

Bugs could arise because `string::find` return -1 when the query is not found.

Eg:

```
for(int i = 0; i < size; i++){
    i = s.find(c, i);
    //More code...
}
```

Correct version:

```
for(int i = 0; i < size; i++){
    i = s.find(c, i);
    if(i == -1) break;
    //More code...
}
```

7.16. Semicolon typo

Eg:

```
for(int i = 0; i < n; i++){
    cout << i << '\n';
}
```

In above code the for loop is prematurely terminated. This would result in runtime error.

7.17. Bugs due to using infinity

Eg:

```
const int inf = numerics::limits<int>max();
int x = inf;
cout << x + inf << '\n';
```

The output here would not be inf. I made a similar mistake while using infinity in Floyd-Warshall's algorithm.

7.18. No code after return

Once I made the following mistake:

```
return r.top(); r.pop();
```

Obviously, that `r.pop()` did not work.

7.19. Beware when using logical operators with functions

The following is dangerous:

```
ans = ans && f(x);
```

If `ans` is initially false, `f` won't be executed.
Better one would be (I am not sure if this always works with all programming languages):

```
ans = f(x) && ans;
```

I made this mistake in [LeetCode 130. Surrounded Regions](#).

7.20. Be careful with char vs int

I made the following mistake:

```
if(mat[i][j] == 1){
// do something.
}
```

Here `mat` was `vector<vector<char>>` I should have tested for `mat[i][j] == '1'`.

7.21. Be careful about pass by reference.

Ref: A student's code.
Not using pass by reference resulted in failure in the update of value of the target. This caused issue in an implementation of linked-list.

7.22. a = a + 1 vs a += 1

Ref: Discussion with a labmate. This is in the context of python programming. `a = a + 1` creates a new object and assigns it to `a`. `a += 1` does not create a new object. It modifies the existing object.

7.23. Failure to initialize all the variables

Ref: A student's code.
The student did not initialize all the variables. This resulted in a bug. This was particularly critical because one of the class variable was a pointer. This resulted in segmentation fault. It also cause TLE, but I am not sure about the exact mechanism.

8. Variable nomenclature for competitive programming

Category	Symbols
Counter	i, j, k; i1, i2
Limits	(h, l), (u, d), (l, r), (h1, l1)
Integers	n, m, k; n1, n2
Floats	x, y, z, x1, x2
Element	e: for(auto e: v)
Pair/points	p, q, r, p1, p2
Pointer	ptr; ptr1, ptr
String	s; s1, s2
Map	mp; mp1, mp2
Vector	a, b, c, u, v; u1, u2, v1, v2
Matrix	mat, mat1, mat2
Graph	g, g1, g2
DP mat/vect	dp; dp1, dp2
Priority queue	pq; pq1; pq2
Stack	stk; stk1, stk2
Set	set1, set2
Temporary	tmp; tmp1, tmp2

Using suffix

Eg:	Description
ai	Element of a
al	Left limit element of a (or) Left limit ptr of a
ar	Right limit element of a (or) Right limit ptr of a
aptr	Pointer to an element in a
ait	Iterator to a

Using prefix

Eg:	Description
na	Number of elements in a
ma	Map associated with a
sa	Set associated with a
qa	Queue or priority queue associated with a
ga	Graph associated with a